

Volume 3 – System Architecture Document (SAD)

AI Quant Trading Platform

Version: 1.0

Document Type: System Architecture Document

1. Purpose

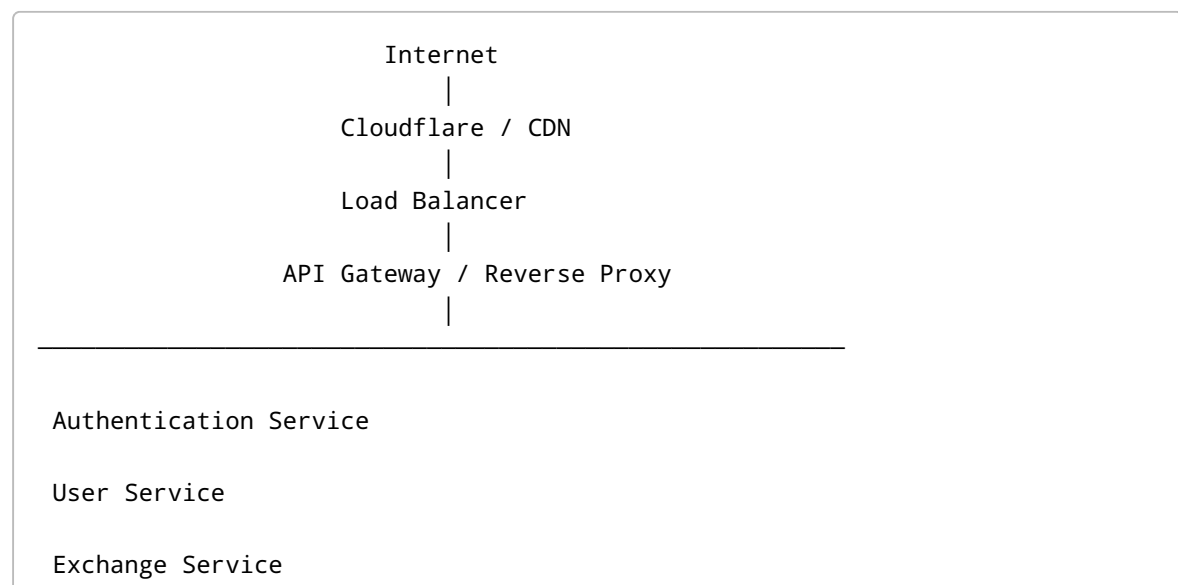
This document defines the complete architecture of the AI Quant Trading Platform.

The system is designed as a cloud-native, event-driven, modular platform capable of continuously scanning cryptocurrency markets, evaluating opportunities, managing risk, and executing automated trades while supporting thousands of concurrent users.

The architecture prioritizes:

- High Availability
 - Scalability
 - Fault Tolerance
 - Low Latency
 - Security
 - Maintainability
 - Observability
-

2. High-Level Architecture



Market Data Service

Scanner Service

Technical Analysis Service

AI Decision Engine

Strategy Engine

Risk Engine

Order Management Service

Portfolio Service

Notification Service

Reporting Service

Billing Service

Admin Service

|
Message Broker (Redis Streams/RabbitMQ)

PostgreSQL

Redis

Object Storage

Time-Series Database (Optional)

Monitoring Stack

Logging Stack

Alerting

Backup

3. Architecture Principles

- Modular Design
 - API First
 - Event Driven
 - Horizontal Scalability
 - Stateless Services
 - Centralized Logging
 - Secure by Default
 - Fail Safe Execution
 - Idempotent Order Processing
-

4. Core Services

Authentication Service

Responsibilities

- Login
- Registration
- JWT
- OAuth
- 2FA
- Session Management

Database

User Database

Communication

REST

User Service

Responsibilities

- Profile
- Preferences
- Subscription
- Settings

Communication

REST

Exchange Service

Responsibilities

- API Key Storage
- Exchange Authentication
- WebSocket Connections
- Order Submission
- Balance Synchronization
- Position Synchronization
- Rate Limit Handling
- Retry Logic

This service is the only component allowed to communicate directly with exchange APIs.

Market Data Service

Responsibilities

Collect

- Price
- Candles
- Trades
- Order Book
- Funding Rate
- Open Interest
- Liquidations

Data Sources

WebSocket

REST

Internal Cache

Redis

Scanner Service

Runs continuously.

Responsibilities

- Scan every supported trading pair
- Remove low-quality opportunities
- Send qualified candidates to AI Engine

Workers

Multiple parallel workers.

Technical Analysis Service

Responsibilities

Calculate

EMA

RSI

MACD

VWAP

ATR

SuperTrend

Volume Profile

Support

Resistance

Trend

Momentum

All indicator calculations are cached for reuse.

AI Decision Engine

Responsibilities

Receive

Scanner Results

Indicator Results

Market Structure

Sentiment

News

Risk Profile

Generate

Probability Score

Confidence Score

Trade Recommendation

Reasoning

The AI engine never sends orders directly.

It only produces recommendations.

Strategy Engine

Responsibilities

Apply

Trading Rules

Entry Rules

Exit Rules

Position Management

Time Filters

Risk Filters

Supported

Trend Following

Swing

Scalping

Breakout

Grid

DCA

Custom Strategies

Risk Engine

The final approval authority before order execution.

Responsibilities

Validate

Maximum Daily Loss

Maximum Position Size

Risk Per Trade

Portfolio Exposure

Drawdown

Consecutive Losses

Maximum Open Positions

Emergency Stop

If any rule fails,

Order is rejected.

Order Management Service

Responsibilities

Create Orders

Modify Orders

Cancel Orders

Track Orders

Retry Failed Orders

Synchronize Exchange Status

Maintain Order History

Portfolio Service

Responsibilities

Balances

PnL

Asset Allocation

Performance

Exposure

History

Notification Service

Channels

Email

Telegram

WhatsApp

Push

Discord

Slack

Triggers

Trade Executed

Position Closed

Risk Warning

Exchange Error

Daily Report

Reporting Service

Generate

Performance Reports

PnL Reports

Tax Reports

Trade Reports

Strategy Reports

PDF Export

CSV Export

Billing Service

Responsibilities

Subscription

Invoices

Coupons

Plan Limits

Payments

Admin Service

Responsibilities

System Monitoring

User Management

Subscription Management

Exchange Monitoring

Log Viewer

Analytics

Feature Flags

5. Data Flow

Market Data

↓

Scanner

↓

Technical Analysis

↓

AI Decision Engine

↓

Strategy Engine

↓

Risk Engine

↓

Order Manager

↓

Exchange

↓

Portfolio Update

↓

Notification

↓

Reports

6. Communication Pattern

REST API

Used For

Authentication

Dashboard

Reports

Settings

WebSocket

Used For

Live Prices

Portfolio Updates

Trade Updates

Notifications

Internal Event Bus

Used For

Scanner Events

AI Events

Risk Events

Order Events

Report Generation

7. Background Workers

Worker Types

Market Scanner

Indicator Calculation

AI Scoring

Order Synchronization

Portfolio Synchronization

Notification Sender

Report Generator

Backtesting

ML Training

Each worker is independently scalable.

8. Database Architecture

Primary Database

PostgreSQL

Caching

Redis

Optional Future

ClickHouse (analytics)

TimescaleDB (time-series)

Object Storage

Backups

Logs

Reports

9. Deployment Architecture

Production

Internet

↓

Cloudflare

↓

Load Balancer

↓

NGINX

↓

Docker Containers

↓

Application Cluster

↓

Redis

↓

PostgreSQL

↓

Monitoring

All services should run in isolated containers.

10. Logging

Every service logs:

Request ID

User ID

Exchange ID

Strategy ID

Execution Time

Errors

Warnings

Security Events

11. Monitoring

Metrics

CPU

RAM

Disk

Queue Size

API Latency

Scanner Speed

Trade Execution Time

Exchange Latency

Worker Status

12. Security Architecture

Authentication

JWT

Authorization

RBAC

Encryption

AES-256 for stored secrets

TLS for communication

Audit

Complete activity logs

Rate Limiting

Enabled

IP Restrictions

Supported

13. Fault Tolerance

Automatic Retry

Circuit Breaker

Dead Letter Queue

Health Checks

Graceful Shutdown

Worker Recovery

Exchange Reconnection

Database Failover (future)

14. Scalability

Horizontal Scaling

Scanner Workers

AI Workers

Notification Workers

REST APIs

WebSocket Nodes

The architecture should support scaling individual services independently without affecting other components.

15. Future Architecture Enhancements

- Kubernetes orchestration
 - Multi-region deployment
 - Active-active failover
 - AI model serving cluster
 - GPU inference nodes
 - Feature store for machine learning
 - Multi-cloud deployment
 - Cross-exchange smart routing
 - Real-time anomaly detection
 - Reinforcement learning experimentation
-

16. Design Rules

- No service accesses another service's database directly.
- All inter-service communication must occur through APIs or the event bus.
- Order execution must pass through the Risk Engine before reaching the Exchange Service.
- AI recommendations are advisory; execution always depends on configured strategy and risk policies.
- Every critical action must generate an audit log.
- Every service must expose health-check endpoints.
- All long-running tasks must execute asynchronously.